EXPERIMENT 5: K-NEAREST NEIGHBOURS (K-NN)

Experiment Question: Write a program for Implementation of K-Nearest Neighbours (K-NN) in Python.

K-NN: General concept and mathematical rule

K-Nearest Neighbours is a simple instance-based (lazy) learning algorithm. It does not build a model in advance; it stores all training samples. To classify a new sample, it finds the K training samples closest to it and assigns the class that occurs most often among them.

Distance measure: the Euclidean distance between $p = (p_1, \ldots, p_n)$ and $q = (q_1, \ldots, q_n)$ is $d(p, q) = \sqrt{[(p_1 - q_1)^2 + \ldots + (p_n - q_n)^2]}$.

Choosing K: a small K is sensitive to noise, while a large K smooths the decision but may include points of other classes. An odd K avoids ties in two-class problems; in this program a tie is resolved in favour of the nearest neighbour.

Feature scaling: because distances add squared differences, a feature with large values dominates the distance. Features measured on different scales should be normalised first, for example by min–max scaling $x' = (x - \min)/(\max - \min)$.

Important: the results describe the training examples used here and the chosen K; Subproblem 3 loads the Iris dataset from scikit-learn (pip install scikit-learn).

Subproblem 1: Paper Tissue Quality Classification

1. Aim

To implement K-NN to classify a paper tissue as Good or Bad from its acid durability and strength.

2. Concept

Each tissue is a point in two-dimensional space (acid durability, strength). The query tissue receives the majority class of its K = 3 nearest training tissues.

3. Dataset Description

The training examples are shown below. The last column is the class label.

Acid durability (s)	Strength (kg/m²)	Class
7	7	Bad
7	4	Bad
3	4	Good
1	4	Good

Query sample: (3, 7) with K = 3.

4. Algorithm

1. Store the training samples with their class labels and choose the value of K.
2. Take the query (new) sample whose class is to be predicted.
3. Compute the Euclidean distance from the query to every training sample.
4. Sort the distances in ascending order and select the K nearest training samples.
5. Count the class labels among the K neighbours.
6. Assign the majority class to the query; if there is a tie, the class of the nearest neighbour wins.
7. When a test set is used, repeat steps 2–6 for each test sample and calculate the accuracy.

5. Step-by-step Procedure and Detailed Calculations

1. Query Q = (3, 7), K = 3.
2. $d(Q, P1) = \sqrt{((7-3)^2 + (7-7)^2)} = \sqrt{16} = 4.0000$ [Bad]
3. $d(Q, P2) = \sqrt{((7-3)^2 + (4-7)^2)} = \sqrt{25} = 5.0000$ [Bad]
4. $d(Q, P3) = \sqrt{((3-3)^2 + (4-7)^2)} = \sqrt{9} = 3.0000$ [Good]
5. $d(Q, P4) = \sqrt{((1-3)^2 + (4-7)^2)} = \sqrt{13} = 3.6056$ [Good]
6. Sorted by distance (nearest first): P3 (3.0000, Good), P4 (3.6056, Good), P1 (4.0000, Bad), P2 (5.0000, Bad).
7. The K = 3 nearest neighbours are P3 (Good), P4 (Good), P1 (Bad).
8. Votes: Good = 2, Bad = 1 → predicted class Good.

6. Formulae

Let $p = (p_1, \ldots, p_n)$ be a training sample and $q = (q_1, \ldots, q_n)$ the query.

- Euclidean distance: $d(p, q) = \sqrt{[\ (p_1 - q_1)^2 + (p_2 - q_2)^2 + \ldots + (p_n - q_n)^2\ ]}$
- Prediction: the class that occurs most often among the K smallest distances
- Min–max normalisation: $x' = (x - \min) / (\max - \min)$
- Accuracy = (number of correct predictions) / (number of test samples) × 100

7. Python Program

```
import math
from collections import Counter

def euclidean(a, b):
    return math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))

def knn_predict(train_X, train_y, query, k):
    ranked = sorted((euclidean(x, query), label) for x, label in zip(train_X,
train_y))
    neighbours = ranked[:k]                                    # k nearest
points
    votes = Counter(label for _, label in neighbours)          # majority vote
    return votes.most_common(1)[0][0], neighbours

# Paper tissue: [acid durability (secs), strength (kg/m2)] -> quality
X = [[7, 7], [7, 4], [3, 4], [1, 4]]
y = ['Bad', 'Bad', 'Good', 'Good']

query, k = [3, 7], 3
label, neighbours = knn_predict(X, y, query, k)

print("Query point:", query, "  K =", k)
print("\nDistances to all training points")
for x, cls in zip(X, y):
    print("  ", x, " ->  %.4f  (%s)" % (euclidean(x, query), cls))
print("\nNearest neighbours:", [(round(d, 4), c) for d, c in neighbours])
print("Predicted class:", label)
```

8. Output

```
>>>
===== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/5.1.py ====
Query point: [3, 7]   K = 3

Distances to all training points
   [7, 7]  ->  4.0000  (Bad)
   [7, 4]  ->  5.0000  (Bad)
   [3, 4]  ->  3.0000  (Good)
   [1, 4]  ->  3.6056  (Good)

Nearest neighbours: [(3.0, 'Good'), (3.6056, 'Good'), (4.0, 'Bad')]
Predicted class: Good
>>>
```

9. Result / Conclusion

K-NN was implemented successfully. With K = 3, the query (3, 7) is classified as Good.

Subproblem 2: Student Result Prediction

1. Aim

To use K-NN to predict whether a student will Pass or Fail from daily study hours and attendance.

2. Concept

Each student is a point (study hours, attendance %). The attendance values are much larger than the study hours, so attendance dominates the distance here; this is discussed in Subproblem 4.

3. Dataset Description

The training examples are shown below. The last column is the class label.

Study hours/day	Attendance (%)	Result
2	50	Fail
3	60	Fail
4	55	Fail
5	85	Pass
6	80	Pass
7	90	Pass
3	85	Fail
8	95	Pass

Query sample: (5, 70) with K = 3.

4. Algorithm

1. Store the training samples with their class labels and choose the value of K.
2. Take the query (new) sample whose class is to be predicted.
3. Compute the Euclidean distance from the query to every training sample.
4. Sort the distances in ascending order and select the K nearest training samples.
5. Count the class labels among the K neighbours.
6. Assign the majority class to the query; if there is a tie, the class of the nearest neighbour wins.
7. When a test set is used, repeat steps 2–6 for each test sample and calculate the accuracy.

5. Step-by-step Procedure and Detailed Calculations

1. Query Q = (5, 70), K = 3.
2. $d(Q, P1) = \sqrt{((2-5)^2 + (50-70)^2)} = \sqrt{409} = 20.2237$ [Fail]
3. $d(Q, P2) = \sqrt{((3-5)^2 + (60-70)^2)} = \sqrt{104} = 10.1980$ [Fail]
4. $d(Q, P3) = \sqrt{((4-5)^2 + (55-70)^2)} = \sqrt{226} = 15.0333$ [Fail]
5. $d(Q, P4) = \sqrt{((5-5)^2 + (85-70)^2)} = \sqrt{225} = 15.0000$ [Pass]
6. $d(Q, P5) = \sqrt{((6-5)^2 + (80-70)^2)} = \sqrt{101} = 10.0499$ [Pass]

7. $d(Q, P6) = \sqrt{((7-5)^2 + (90-70)^2)} = \sqrt{404} = 20.0998$ [Pass]
8. $d(Q, P7) = \sqrt{((3-5)^2 + (85-70)^2)} = \sqrt{229} = 15.1327$ [Fail]
9. $d(Q, P8) = \sqrt{((8-5)^2 + (95-70)^2)} = \sqrt{634} = 25.1794$ [Pass]
10. Sorted by distance (nearest first): P5 (10.0499, Pass), P2 (10.1980, Fail), P4 (15.0000, Pass), P3 (15.0333, Fail), P7 (15.1327, Fail) …
11. The K = 3 nearest neighbours are P5 (Pass), P2 (Fail), P4 (Pass).
12. Votes: Pass = 2, Fail = 1 → predicted class Pass.

6. Formulae

Let $p = (p_1, \ldots, p_n)$ be a training sample and $q = (q_1, \ldots, q_n)$ the query.

- Euclidean distance: $d(p, q) = \sqrt{[\ (p_1 - q_1)^2 + (p_2 - q_2)^2 + \ldots + (p_n - q_n)^2\ ]}$
- Prediction: the class that occurs most often among the K smallest distances
- Min–max normalisation: $x' = (x - \min) / (\max - \min)$
- Accuracy = (number of correct predictions) / (number of test samples) × 100

7. Python Program

```
import math
from collections import Counter

def euclidean(a, b):
    return math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))

def knn_predict(train_X, train_y, query, k):
    ranked = sorted((euclidean(x, query), label) for x, label in zip(train_X,
train_y))
    neighbours = ranked[:k]                                    # k nearest
points
    votes = Counter(label for _, label in neighbours)          # majority vote
    return votes.most_common(1)[0][0], neighbours

# Student result: [study hours per day, attendance %] -> Pass / Fail
X = [[2, 50], [3, 60], [4, 55], [5, 85], [6, 80], [7, 90], [3, 85], [8, 95]]
y = ['Fail', 'Fail', 'Fail', 'Pass', 'Pass', 'Pass', 'Fail', 'Pass']

query, k = [5, 70], 3
label, neighbours = knn_predict(X, y, query, k)

print("Query student (hours, attendance):", query, "  K =", k)
print("\nDistances to all training points")
for x, cls in zip(X, y):
    print("  ", x, " ->  %.4f  (%s)" % (euclidean(x, query), cls))
print("\nNearest neighbours:", [(round(d, 4), c) for d, c in neighbours])
print("Predicted result:", label)
```

8. Output

```
>>>
===== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/5.2.py ====
Query student (hours, attendance): [5, 70]   K = 3

Distances to all training points
   [2, 50]  ->  20.2237  (Fail)
   [3, 60]  ->  10.1980  (Fail)
   [4, 55]  ->  15.0333  (Fail)
   [5, 85]  ->  15.0000  (Pass)
   [6, 80]  ->  10.0499  (Pass)
   [7, 90]  ->  20.0998  (Pass)
   [3, 85]  ->  15.1327  (Fail)
   [8, 95]  ->  25.1794  (Pass)

Nearest neighbours: [(10.0499, 'Pass'), (10.198, 'Fail'), (15.0, 'Pass')]
Predicted result: Pass
>>>
```

9. Result / Conclusion

K-NN was implemented successfully. With K = 3, the query (5, 70) is classified as Pass.

Subproblem 3: Iris Species Classification with Choice of K

1. Aim

To implement K-NN on the Iris dataset, compare the accuracy for several values of K, and classify a new flower.

2. Concept

The data is split randomly into 120 training and 30 test samples. The program tests K = 1, 3, 5, 7, 9 and uses the K with the best test accuracy to classify a new flower.

3. Dataset Description

The Iris dataset (built into scikit-learn) has 150 samples, 4 numeric features, and 3 species with 50 samples each. Six representative rows are shown below.

Sepal length	Sepal width	Petal length	Petal width	Species
5.1	3.5	1.4	0.2	setosa
4.9	3	1.4	0.2	setosa
7	3.2	4.7	1.4	versicolor
6.4	3.2	4.5	1.5	versicolor
6.3	3.3	6	2.5	virginica
5.8	2.7	5.1	1.9	virginica

New sample to classify: [5.9, 3.0, 5.1, 1.8] (sepal length, sepal width, petal length, petal width in cm).

4. Algorithm

1. Store the training samples with their class labels and choose the value of K.
2. Take the query (new) sample whose class is to be predicted.
3. Compute the Euclidean distance from the query to every training sample.
4. Sort the distances in ascending order and select the K nearest training samples.
5. Count the class labels among the K neighbours.
6. Assign the majority class to the query; if there is a tie, the class of the nearest neighbour wins.
7. When a test set is used, repeat steps 2–6 for each test sample and calculate the accuracy.

5. Step-by-step Procedure and Detailed Calculations

1. The 150 Iris samples are shuffled (seed 11) and split into 120 training and 30 test samples.
2. For each test flower the program computes $d = \sqrt{\Sigma(x_i - q_i)^2}$ over the four features to all 120 training flowers, picks the K nearest, and takes the majority species.
3. Result: K = 1 correct = 29 / 30 accuracy = 96.67%.
4. Result: K = 3 correct = 29 / 30 accuracy = 96.67%.
5. Result: K = 5 correct = 28 / 30 accuracy = 93.33%.
6. Result: K = 7 correct = 29 / 30 accuracy = 96.67%.

7. Result: K = 9 correct = 29 / 30 accuracy = 96.67%.
8. Best K = 1 (first K with the highest accuracy). For the new flower [5.9, 3.0, 5.1, 1.8] the nearest neighbour distance(s) are: 0.2828 (virginica).
9. Majority species among the 1 nearest neighbour(s): virginica.

6. Formulae

Let $p = (p_1, \ldots, p_n)$ be a training sample and $q = (q_1, \ldots, q_n)$ the query.

- Euclidean distance: $d(p, q) = \sqrt{[\ (p_1 - q_1)^2 + (p_2 - q_2)^2 + \ldots + (p_n - q_n)^2\ ]}$
- Prediction: the class that occurs most often among the K smallest distances
- Min–max normalisation: $x' = (x - \min) / (\max - \min)$
- Accuracy = (number of correct predictions) / (number of test samples) × 100

7. Python Program

```
import math
from collections import Counter

def euclidean(a, b):
    return math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))

def knn_predict(train_X, train_y, query, k):
    ranked = sorted((euclidean(x, query), label) for x, label in zip(train_X,
train_y))
    neighbours = ranked[:k]                                    # k nearest
points
    votes = Counter(label for _, label in neighbours)          # majority vote
    return votes.most_common(1)[0][0], neighbours
import random
from sklearn.datasets import load_iris

iris = load_iris()
data = list(zip(iris.data.tolist(), [iris.target_names[t] for t in
iris.target]))
random.seed(11)
random.shuffle(data)
train, test = data[:120], data[120:]                    # 120 train / 30 test
train_X, train_y = [d[0] for d in train], [d[1] for d in train]

print("Accuracy of K-NN on the Iris test set")
best_k, best_acc = 1, 0
for k in [1, 3, 5, 7, 9]:
    correct = sum(knn_predict(train_X, train_y, x, k)[0] == actual for x,
actual in test)
    acc = correct / len(test) * 100
    print("  K = %d   correct = %2d / %d   accuracy = %.2f%%" % (k, correct,
len(test), acc))
    if acc > best_acc:
        best_k, best_acc = k, acc

sample = [5.9, 3.0, 5.1, 1.8]                              # new flower
label, neighbours = knn_predict(train_X, train_y, sample, best_k)
print("\nBest K =", best_k)
print("New sample", sample, "-> predicted species:", label)
```

8. Output

```
>>>
===== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/5.3.py ====
Accuracy of K-NN on the Iris test set
  K = 1   correct = 29 / 30   accuracy = 96.67%
  K = 3   correct = 29 / 30   accuracy = 96.67%
  K = 5   correct = 28 / 30   accuracy = 93.33%
  K = 7   correct = 29 / 30   accuracy = 96.67%
  K = 9   correct = 29 / 30   accuracy = 96.67%

Best K = 1
New sample [5.9, 3.0, 5.1, 1.8] -> predicted species: virginica
>>>
```

9. Result / Conclusion

K-NN was implemented successfully on the Iris data. Test accuracy was 96.67% for K = 1, 96.67% for K = 3, 93.33% for K = 5, 96.67% for K = 7, 96.67% for K = 9. The new flower [5.9, 3.0, 5.1, 1.8] is classified as virginica using K = 1.

Subproblem 4: Customer Purchase Prediction with Normalisation

1. Aim

To show how min–max normalisation changes the K-NN result when features are on different scales (age in years, salary in thousands).

2. Concept

Salary values are larger than age values, so unscaled distances are dominated by salary. Min–max normalisation rescales each feature to 0–1 before distances are computed.

3. Dataset Description

The training examples are shown below. The last column is the class label.

Age (years)	Salary (₹ thousand)	Buys product
22	25	No
25	32	No
28	30	No
30	45	No
35	60	Yes
38	52	No
42	80	Yes
47	70	Yes
52	90	Yes
56	65	No

Query customer: age 45, salary 50 thousand, K = 3.

4. Algorithm

1. Store the training samples with their class labels and choose the value of K.
2. Take the query (new) sample whose class is to be predicted.
3. Compute the Euclidean distance from the query to every training sample.
4. Sort the distances in ascending order and select the K nearest training samples.
5. Count the class labels among the K neighbours.
6. Assign the majority class to the query; if there is a tie, the class of the nearest neighbour wins.
7. When a test set is used, repeat steps 2–6 for each test sample and calculate the accuracy.

5. Step-by-step Procedure and Detailed Calculations

1. Query Q = (45, 50) (age, salary in thousands), K = 3.
2. Feature ranges from the training data: age 22–56, salary 25–90.
3. Without normalisation, the nearest points are P6 (7.2801, No), P5 (14.1421, Yes), P4 (15.8114, No) → No.
4. Normalised query: $Q' = ((45-22)/(56-22), (50-25)/(90-25)) = (0.6765, 0.3846)$.

5. $d(Q, P1) = \sqrt{((0-0.6765)^2 + (0-0.3846)^2)} = \sqrt{0.6055} = 0.7782$ [No]
6. $d(Q, P2) = \sqrt{((0.0882-0.6765)^2 + (0.1077-0.3846)^2)} = \sqrt{0.4227} = 0.6502$ [No]
7. $d(Q, P3) = \sqrt{((0.1765-0.6765)^2 + (0.0769-0.3846)^2)} = \sqrt{0.3447} = 0.5871$ [No]
8. $d(Q, P4) = \sqrt{((0.2353-0.6765)^2 + (0.3077-0.3846)^2)} = \sqrt{0.2006} = 0.4478$ [No]
9. $d(Q, P5) = \sqrt{((0.3824-0.6765)^2 + (0.5385-0.3846)^2)} = \sqrt{0.1102} = 0.3319$ [Yes]
10. $d(Q, P6) = \sqrt{((0.4706-0.6765)^2 + (0.4154-0.3846)^2)} = \sqrt{0.0433} = 0.2082$ [No]
11. $d(Q, P7) = \sqrt{((0.5882-0.6765)^2 + (0.8462-0.3846)^2)} = \sqrt{0.2208} = 0.4699$ [Yes]
12. $d(Q, P8) = \sqrt{((0.7353-0.6765)^2 + (0.6923-0.3846)^2)} = \sqrt{0.0981} = 0.3133$ [Yes]
13. $d(Q, P9) = \sqrt{((0.8824-0.6765)^2 + (1-0.3846)^2)} = \sqrt{0.4211} = 0.6489$ [Yes]
14. $d(Q, P10) = \sqrt{((1-0.6765)^2 + (0.6154-0.3846)^2)} = \sqrt{0.1579} = 0.3974$ [No]
15. Sorted by distance (nearest first): P6 (0.2082, No), P8 (0.3133, Yes), P5 (0.3319, Yes), P10 (0.3974, No), P4 (0.4478, No) …
16. The K = 3 nearest neighbours are P6 (No), P8 (Yes), P5 (Yes).
17. Votes: Yes = 2, No = 1 → predicted class Yes.

6. Formulae

Let $p = (p_1, \ldots, p_n)$ be a training sample and $q = (q_1, \ldots, q_n)$ the query.

- Euclidean distance: $d(p, q) = \sqrt{[\ (p_1 - q_1)^2 + (p_2 - q_2)^2 + \ldots + (p_n - q_n)^2\ ]}$
- Prediction: the class that occurs most often among the K smallest distances
- Min–max normalisation: $x' = (x - \min) / (\max - \min)$
- Accuracy = (number of correct predictions) / (number of test samples) × 100

7. Python Program

```
import math
from collections import Counter

def euclidean(a, b):
    return math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))

def knn_predict(train_X, train_y, query, k):
    ranked = sorted((euclidean(x, query), label) for x, label in zip(train_X,
train_y))
    neighbours = ranked[:k]                                    # k nearest
points
    votes = Counter(label for _, label in neighbours)          # majority vote
    return votes.most_common(1)[0][0], neighbours

# Customer purchase: [age, salary in thousands] -> will buy (Yes / No)
X = [[22, 25], [25, 32], [28, 30], [30, 45], [35, 60], [38, 52], [42, 80],
[47, 70], [52, 90], [56, 65]]
y = ['No', 'No', 'No', 'No', 'Yes', 'No', 'Yes', 'Yes', 'Yes', 'No']

def min_max(rows, mins, maxs):
    return [[(v - lo) / (hi - lo) for v, lo, hi in zip(r, mins, maxs)] for r
in rows]

mins = [min(col) for col in zip(*X)]
maxs = [max(col) for col in zip(*X)]
```

```
Xn = min_max(X, mins, maxs)                                    # normalised
training data


query, k = [45, 50], 3
print("Without normalisation:", knn_predict(X, y, query, k)[0])
qn = min_max([query], mins, maxs)[0]
label, neighbours = knn_predict(Xn, y, qn, k)
print("With normalisation   :", label)
print("\nNormalised query:", [round(v, 4) for v in qn])
print("Nearest neighbours (distance, class):", [(round(d, 4), c) for d, c in
neighbours])
```

8. Output

```
>>>
===== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/5.4.py ====
Without normalisation: No
With normalisation   : Yes

Normalised query: [0.6765, 0.3846]
Nearest neighbours (distance, class): [(0.2082, 'No'), (0.3133, 'Yes'), (0.3319,
 'Yes')]
>>>
```

9. Result / Conclusion

K-NN was implemented successfully. Without normalisation the query (45, 50) is classified as No; after min–max normalisation it is classified as Yes. This shows that feature scaling can change the K-NN prediction.

Subproblem 5: T-Shirt Size Classification

1. Aim

To implement K-NN to predict a T-shirt size (M or L) from height and weight, and to observe the effect of changing K.

2. Concept

Each customer is a point (height, weight). The same query is classified using K = 1, 3, 5, and 7 to show that the answer can depend on K.

3. Dataset Description

The training examples (18 customers) are shown below. The last column is the T-shirt size.

Height (cm)	Weight (kg)	Size
158	58	M
158	59	M
158	63	M
160	59	M
160	60	M
163	60	M
163	61	M
160	64	L
163	64	L
165	61	L
165	62	L
165	65	L
168	62	L
168	63	L
168	66	L
170	63	L
170	64	L
170	68	L

Query customer: height 161 cm, weight 61 kg.

4. Algorithm

1. Store the training samples with their class labels and choose the value of K.
2. Take the query (new) sample whose class is to be predicted.
3. Compute the Euclidean distance from the query to every training sample.
4. Sort the distances in ascending order and select the K nearest training samples.
5. Count the class labels among the K neighbours.
6. Assign the majority class to the query; if there is a tie, the class of the nearest neighbour wins.

7. When a test set is used, repeat steps 2–6 for each test sample and calculate the accuracy.

5. Step-by-step Procedure and Detailed Calculations

1. Query Q = (161, 61) (height cm, weight kg).
2. d(Q, P1) = √((158−161)² + (58−61)²) = √18 = 4.2426 [M]
3. d(Q, P2) = √((158−161)² + (59−61)²) = √13 = 3.6056 [M]
4. d(Q, P3) = √((158−161)² + (63−61)²) = √13 = 3.6056 [M]
5. d(Q, P4) = √((160−161)² + (59−61)²) = √5 = 2.2361 [M]
6. d(Q, P5) = √((160−161)² + (60−61)²) = √2 = 1.4142 [M]
7. d(Q, P6) = √((163−161)² + (60−61)²) = √5 = 2.2361 [M]
8. d(Q, P7) = √((163−161)² + (61−61)²) = √4 = 2.0000 [M]
9. d(Q, P8) = √((160−161)² + (64−61)²) = √10 = 3.1623 [L]
10. d(Q, P9) = √((163−161)² + (64−61)²) = √13 = 3.6056 [L]
11. d(Q, P10) = √((165−161)² + (61−61)²) = √16 = 4.0000 [L]
12. d(Q, P11) = √((165−161)² + (62−61)²) = √17 = 4.1231 [L]
13. d(Q, P12) = √((165−161)² + (65−61)²) = √32 = 5.6569 [L]
14. d(Q, P13) = √((168−161)² + (62−61)²) = √50 = 7.0711 [L]
15. d(Q, P14) = √((168−161)² + (63−61)²) = √53 = 7.2801 [L]
16. d(Q, P15) = √((168−161)² + (66−61)²) = √74 = 8.6023 [L]
17. d(Q, P16) = √((170−161)² + (63−61)²) = √85 = 9.2195 [L]
18. d(Q, P17) = √((170−161)² + (64−61)²) = √90 = 9.4868 [L]
19. d(Q, P18) = √((170−161)² + (68−61)²) = √130 = 11.4018 [L]
20. Sorted by distance: P5 (1.4142, M), P7 (2.0000, M), P4 (2.2361, M), P6 (2.2361, M), P8 (3.1623, L), P9 (3.6056, L), P2 (3.6056, M) …
21. K = 1: neighbours ['M'] → M = 1 → M.
22. K = 3: neighbours ['M', 'M', 'M'] → M = 3 → M.
23. K = 5: neighbours ['M', 'M', 'M', 'M', 'L'] → M = 4, L = 1 → M.
24. K = 7: neighbours ['M', 'M', 'M', 'M', 'L', 'L', 'M'] → M = 5, L = 2 → M.

6. Formulae

Let $p = (p_1, \ldots, p_n)$ be a training sample and $q = (q_1, \ldots, q_n)$ the query.

- Euclidean distance: $d(p, q) = \sqrt{[\ (p_1 - q_1)^2 + (p_2 - q_2)^2 + \ldots + (p_n - q_n)^2\ ]}$
- Prediction: the class that occurs most often among the K smallest distances
- Min–max normalisation: $x' = (x - \min) / (\max - \min)$
- Accuracy = (number of correct predictions) / (number of test samples) × 100

7. Python Program

```
import math
from collections import Counter
```

```
def euclidean(a, b):
    return math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))

def knn_predict(train_X, train_y, query, k):
    ranked = sorted((euclidean(x, query), label) for x, label in zip(train_X,
train_y))
    neighbours = ranked[:k]                                    # k nearest
points
    votes = Counter(label for _, label in neighbours)          # majority vote
    return votes.most_common(1)[0][0], neighbours

# T-shirt size: [height cm, weight kg] -> size
X = [[158, 58], [158, 59], [158, 63], [160, 59], [160, 60], [163, 60], [163,
61],
     [160, 64], [163, 64], [165, 61], [165, 62], [165, 65], [168, 62], [168,
63],
     [168, 66], [170, 63], [170, 64], [170, 68]]
y = ['M'] * 7 + ['L'] * 11

query = [161, 61]
print("Query (height, weight):", query)
for k in [1, 3, 5, 7]:
    label, neighbours = knn_predict(X, y, query, k)
    print("K = %d  ->  %s    neighbours: %s" % (k, label, [c for _, c in
neighbours]))
```

8. Output

```
>>>
===== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/5.5.py ====
Query (height, weight): [161, 61]
K = 1  ->  M    neighbours: ['M']
K = 3  ->  M    neighbours: ['M', 'M', 'M']
K = 5  ->  M    neighbours: ['M', 'M', 'M', 'M', 'L']
K = 7  ->  M    neighbours: ['M', 'M', 'M', 'M', 'L', 'L', 'M']
>>>
```

9. Result / Conclusion

K-NN was implemented successfully. The query (161, 61) is classified as size M for K = 1, 3, 5, and 7; the vote becomes less clear-cut as K increases because larger-sized neighbours enter the vote.

Overall Result of Experiment 5

K-NN was implemented from scratch and demonstrated on five datasets. The setting and the predicted class for the query sample of each program are summarised below.

Subproblem	Application	Setting	Predicted class
1	Paper Tissue Quality Classification	K = 3, query (3, 7)	Good
2	Student Result Prediction	K = 3, query (5, 70)	Pass
3	Iris Species Classification with Choice of K	best K = 1	virginica
4	Customer Purchase Prediction with Normalisation	K = 3, normalised	Yes
5	T-Shirt Size Classification	K = 1, 3, 5, 7	M

Final conclusion: K-NN classifies a sample by a majority vote of its K nearest training samples. The prediction depends on the distance measure, on the value of K, and on feature scaling, as Subproblems 3 and 4 show. K-NN needs no training time but must compute distances to every training sample for each prediction.

Notation: the outputs shown are the actual outputs of the programs listed in this record for the datasets given here.